Carleton University
Department of Systems and Computer Engineering
SYSC 4001 - Operating Systems

Assignment 1 - Process Lifecycle and CPU Scheduling Simulator

Due: See Brightspace

Prepared By: Ben Earle

Language: C11 on Linux or WSL

Marks: 100 (Programming: 70; Report: 30)

Assignment Overview

You will implement a discrete-time simulator for process lifecycle management and CPU scheduling. The supplied starting code manages CSV input and output handling. Your task is to implement the simulated operating-system behaviour: PCBs, state changes, Ready and Waiting management, dispatching, CPU and I/O bursts, scheduling policies, context-switch overhead, and performance metrics. A test harness is provided with public test cases; your work will also be evaluated by the TAs using hidden tests.

Upon completing this assignment, students are expected to:

1. Explore and learn an existing code-base.
2. Represent processes using PCBs and maintain the New, Ready, Running, Waiting, and Terminated states.
3. Implement FCFS, Round Robin, and preemptive Shortest-Remaining-Time-First (SRTF) scheduling.
4. Model CPU bursts, I/O waits, dispatch overhead, preemption, and idle time.
5. Generate and interpret response, waiting, turnaround, utilization, and throughput metrics.
6. Design controlled workloads that expose tradeoffs among scheduling algorithms.

Part 1 - Process Scheduling Simulator [70 Marks]

Supplied Package

The student package contains the following files and directories:

- **include/scheduler.h:** Public data types and required function signatures.
- **src/main.c:** Command-line processing and simulator startup.
- **src/io.c:** Input validation, deterministic CSV logging, and supplied metric calculation.
- **src/student_scheduler.c:** Core simulator, *your code goes here*.
- **tests/public/:** Sample inputs and authoritative expected outputs.
- **test_harness.py:** Builds your code, executes every public case, and prints unified diffs.
- **Makefile:** Builds every .c file under src/ into the scheduler executable.

You may replace student_scheduler.c and add your own .c and .h files. You must not change the command-line interface, output filenames, CSV headers, supplied enum values, or logging function signatures.

I suggest you start by exploring the given files, particularly the scheduler.h, as it provides several enums and structures you will need to use. This is good practice for your future employment. It will be very unlikely that you start your own C project from scratch, you will likely be dropped into an existing (much more complex) code base. You will need to parse through header files and learn about the existing structures and definitions to integrate your changes.

Workload Input

Every input file uses this exact header:

```
pid,arrival_time,cpu_burst,io_burst,num_cpu_bursts
```

Column	Meaning
pid	Unique process identifier.
arrival_time	Time at which the process enters the system.
cpu_burst	Duration of every CPU burst for this process.
io_burst	Duration of every I/O wait between CPU bursts.
num_cpu_bursts	Number of identical CPU bursts the process must complete.

A process with cpu_burst 10, io_burst 5, and num_cpu_bursts 2 follows this sequence:

```
CPU for 10 -> I/O for 5 -> CPU for 10 -> terminate
```

Its own CPU and I/O service totals 25 time units. Its completion time may be later because it can also spend time in the Ready state and in dispatch/context-switch intervals.

Required Simulation Model

Time and Devices

- Time is an integer. Work during time t occupies the half-open interval [t, t + 1).
- The system contains one CPU.
- I/O devices are unlimited. Waiting processes perform I/O concurrently and never queue for a device.
- A process returns to Ready exactly io_burst time units after completing a non-final CPU burst.
- The simulation ends when every process is Terminated.

Context-Switch and Dispatch Cost

The command-line context-switch parameter is a nonnegative duration. This cost is paid before every Ready-to-Running dispatch, including the first dispatch and redispatch of the same PID after a Round Robin quantum expires. No process executes during this interval.

> **One simplification:** A context switch does not reserve a process. Arrivals and I/O completions that occur while the switch is in progress enter the Ready queue normally. When the switch duration finishes, the scheduling algorithm may select a process and dispatch it.

Required Ordering at Each Time Step

Be careful to follow this ordering exactly so that your output matches what is expected by the test harness. At each time t, complete the following steps in order:

1. Resolve the CPU interval that ended at t. A completed burst blocks or terminates; an expired RR quantum returns the process to Ready.
2. Admit processes whose arrival_time equals t, in ascending PID order.
3. Complete I/O operations whose completion time equals t, in ascending PID order.
4. For SRTF, determine whether the current process must be preempted.
5. Complete or begin dispatch/context-switch activity as required, then execute one CPU, context-switch, or idle time unit.

Process States and PCB Requirements

State	Required Meaning
NEW	The process has not reached its arrival time.
READY	The process can be executed and is waiting for dispatch.
RUNNING	The process owns the CPU during the current interval.
WAITING	The process is completing an I/O burst.
TERMINATED	All CPU bursts are complete.

Your PCB design must contain enough information to make every scheduling decision. At minimum, track the input fields, state, remaining time in the current CPU burst, completed CPU bursts, I/O completion time, queue-order information, and RR quantum usage.

You must maintain an explicit Ready queue or an equivalent data structure that preserves the specified order. Waiting processes should be maintained in a separate collection.

Scheduling Algorithms

First-Come, First-Served (FCFS): FCFS is non-preemptive. Dispatch the process that has waited in the Ready queue longest. Once running, it continues until its CPU burst completes.

Round Robin (RR): RR uses a FIFO Ready queue and the positive quantum supplied with -q. A process runs until its CPU burst completes or it has consumed one full quantum. On quantum expiry, move it to the tail of the Ready queue and reset its quantum counter when it is next dispatched. A CPU burst that ends exactly at the quantum boundary completes normally and is not logged as a quantum expiry.

Preemptive Shortest Remaining Time First (SRTF): SRTF selects the Ready process with the least remaining time in its current CPU burst. Reevaluate the choice after arrivals and I/O completions. Preempt only when a Ready process has strictly less remaining burst time than the Running process. Equal remaining times do not cause preemption. Break selection ties by Ready-entry order, then by lower PID.

Required CSV Outputs

The supplied OutputContext functions create and format four files. Use log_transition() for every state change and record_timeline_tick() once per simulated interval (time tick). After all processes terminate, call write_metrics(output, specs, count) once.

- **transitions.csv:** Every process state transition, its timestamp, and reason.
- **timeline.csv:** Coalesced intervals of CPU execution, context switching, and idle time.
- **metrics.csv:** One row per PID, in ascending PID order.
- **summary.csv:** One row of whole-workload metrics and averages.

Transition reasons: Use ARRIVAL, DISPATCH, CPU_BURST_COMPLETE, IO_COMPLETE, QUANTUM_EXPIRED, SRTF_PREEMPTION, and PROCESS_COMPLETE exactly as supplied.

Timeline recording: Call record_timeline_tick() exactly once for every simulated time interval. Pass ACTIVITY_CPU and the running PID, ACTIVITY_CONTEXT_SWITCH and PID 0, or ACTIVITY_IDLE and PID 0. The supplied function merges adjacent identical intervals.

Metrics

The supplied write_metrics() function calculates the following values:

- **Completion time:** Timestamp of the transition to Terminated.
- **Turnaround time:** completion_time - arrival_time.

- **Waiting time:** Number of intervals the process spends in Ready, including dispatch overhead.
- **Response time:** first_running_time - arrival_time.
- **Makespan:** Time at which the final process terminates.
- **CPU utilization:** 100 x cpu_busy_time / makespan.
- **Throughput:** process_count / makespan, in processes per time unit.
- **Averages:** Arithmetic mean across all submitted processes.

Build, Run, and Test

Build the simulator from the student-package directory:

```
make clean
make
```

Run one scenario:

```
./scheduler -i tests/public/fcfs_io/input.csv -a fcfs -q 2 -c
1 -o output
```

All five options are required. The quantum must be positive and affects Round Robin only; the context-switch duration must be nonnegative.

Run all public tests:

```
python3 test_harness.py
```

Run one specific test case:

```
python3 test_harness.py --tests tests/public/fcfs_io/
```

The harness compiles the program, runs each case in an isolated temporary directory, and compares all four CSV files. A failure includes a unified diff: lines beginning with - are expected but missing or different, while lines beginning with + came from your program.

Suggested Development Order

Learning to take large problems and decompose them into manageable chunks is an important skill for all engineers! To assist you with this first assignment I have a recommended decomposition (this is how I approached it when creating a sample solution).

1. Design the PCB and initialize one PCB per ProcessSpec. Create helper functions to print the contents of a PCB, this will come in handy later.
2. Define a time variable and a main simulation loop.
3. Create functions to handle each stage of the simulation loop and lay them out in the correct order. I gave you a list of operations in order in a comment in the starting code – consider a helper function for each step.
4. Implement each helper function one at a time. I used simple print debugging and a basic public test scenario like fcfs_io.
5. Focus on FCFS until you can pass its public tests
6. Add context-switch intervals, including events that occur during the switch.

7. Call write metrics and the per tick update using the provided template (By now you will have ~half the marks).
8. Add RR with quantum accounting and queue rotation.
9. Add SRTF with preemption.
10. Complete the experimental report section.

Part 2 - Experimental Report [30 Marks]

Complete the two experiments below to explore scheduling tradeoffs. Choose your own process arrival times and burst lengths, following the requirements for each experiment.

Submit:

- A report of no more than two pages, including your results tables and commands.
- Three input files in report_scenarios/: scheduling.csv, overlap.csv, and baseline.csv.

Keep it brief, no need for an intro/conclusion, just explain the requested tradeoff. You may try different workloads while experimenting, but include only the six required runs in your report.

A. Scheduling Tradeoffs [15 Marks]

Create scheduling.csv with 3–5 processes, each with one CPU burst. Have a long process arrive at time 0, followed by shorter processes that arrive while the long process is running.

Run this workload four times with different scheduling algorithms, keeping the context-switch cost at 1:

Run	Algorithm	RR quantum
1	FCFS	-
2	SRTF	-
3	RR	1
4	RR	Equal to the longest CPU burst in your workload

The command requires -q for every algorithm; use -q 1 for FCFS and SRTF.

Include your four commands and a results table showing:

- Makespan
- Average response time
- Average turnaround time
- Turnaround time of the long process

In no more than 200 words, answer:

1. How does changing the RR quantum affect response time and time spent context switching?

2. How does SRTF compare with FCFS in average turnaround time? What happens to the long process?

Use at least one specific interval from timeline.csv to support your explanation. Explain the results you obtained, even if they differ from what you expected.

B. CPU and I/O Overlap [15 Marks]

Create overlap.csv with two processes:

- One process with at least two CPU bursts and a I/O wait between them.
- A second process that can use the CPU while the first waits for I/O.

Make a copy called baseline.csv and remove the second process. Keep the first process unchanged.

Run both files using FCFS, context-switch cost 1.

Include your two commands and a results table showing:

- CPU busy time
- Idle time
- Makespan
- CPU utilization

In no more than 100 words, answer:

1. When does the second process use CPU time that would otherwise be idle? Give one specific interval from timeline.csv.

2. Why can CPU utilization improve without reducing the makespan?

Marking

Each experiment is worth 15 marks:

Component	**Marks**
Workload follows the requirements and provides a fair comparison	3
Results table and commands	4
Explanation of the tradeoffs	6
Relevant timeline evidence	2

Please try to keep your answers concise.

Submission Requirements

Submit one ZIP archive named SYSC4001_A1_<student-number>.zip containing:

- Makefile, include/, and src/ with all source files required to build your scheduler.
- Your scheduling.csv, overlap.csv, and baseline.csv input files in report_scenarios/.
- Your experimental report as report.pdf.
- A short README.txt if there are any key implementation details to share with your marker.

Marking Rubric

Public and hidden tests provide strong evidence of correctness, but marks are assigned by rubric category. When an integration test fails, the TA will inspect the relevant code and outputs to award part marks for completed components. Code that does not compile can still receive limited implementation marks when the attempted logic is clear.

Category	Marks	Full-Credit Evidence
Build and interface	5	Builds with the supplied Makefile; preserves the CLI and logging API; produces all required files.
PCB, clock, and burst lifecycle	10	Correct initialization, states, repeated CPU/I/O bursts, completion, termination, and simulation progress.
Ready/waiting management	10	Correct queue contents, event order, same-time PID order, I/O returns, and deterministic ties.
FCFS	10	Correct non-preemptive selection, dispatch, state transitions, and results.
Round Robin	10	Correct quantum accounting, expiry, FIFO rotation, boundary behaviour, and redispatch.
Preemptive SRTF	10	Correct remaining-burst selection, arrival/I/O preemption, strict-less-than rule, and tie handling.
Context switching and logs	10	Correct adjustable overhead, reselection after switch events, timeline intervals, and transition calls.
Metrics interface	5	Complete transition/timeline observations and exactly one successful write_metrics() call.
Report Part A	15	See Part 2 for mark breakdown.
Report Part B	15	See Part 2 for mark breakdown.

Total: 100 marks

Statement on AI Use

I used ChatGPT/Codex and GitHub Copilot to help prepare this assignment handout, starter code, test harness, and test cases. The assignment is based on operating-systems exercises I taught in previous courses. I reviewed and tested the materials, completed the programming assignment myself, and take responsibility for their content and correctness.

The supplied code handles input/output processing and metric calculations so that you can focus on process management and scheduling. The public tests provide feedback before submission and help you check your interpretation of the simulation rules. This should give you a useful starting point for testing your own implementation.

Your role in this assignment is to develop and demonstrate those programming and operating-systems skills. Please follow the course's AI-use policy and make sure you understand and can explain every part of the code you submit. Using a tool to help you understand a problem is different from having completed the assessed work for you.

If you find an error, an unclear requirement in the supplied materials, or have any other feedback, please let me know. I am always trying to improve the learning experience for my students.